Gate Analysis: handover and onboarding guide

Oct 4, 2026 · @Janhavee

What this project is, and what you are receiving

Gate Analysis is a Python pipeline that checks a dietary supplement against five "gates" (formulation, UK regulation, lab quality, brand integrity, medical fit) and stops the product at the first gate it fails.

The design rule to remember: **an AI model may only read label text and extract fields; fixed rules make every pass or fail decision.** Everything else in the code follows from that.

What you are receiving

- The pipeline code, tests and demo script, in a private GitHub repository.
- The data tables the rules read: an evidence file of 101 ingredients and several rule tables.
- A small Flask website that runs the pipeline on one product at a time.

Honest status. This is a partially validated research prototype, not a production system. It runs end to end and was compared with about 3,100 hand-labelled rows, but four of the five gates were tested only in part, and Gate 2 could not be tested because no open data exists. Its verdicts are screening aids, not medical or legal endorsements. The validation section has the numbers, and the known-issues section the problems.

Repository map

The pipeline lives under `src/`, the website under `webapp/`, and the data-preparation and validation scripts in the project root; check this table against the repository when you clone it.

Path	What it is	Notes
`run_demo.py`	Runs 21 hand-built product cases through all five gates and writes `output/CN_Automated_Output.xlsx` with a Demo Guide sheet	Needs internet and a Companies House key
`src/pipeline.py`	Runs Gate 0 to 4 in order, stops at the first fail, assigns the tier	The orchestrator
`src/schemas.py`, `src/report_writer.py`	Data models; writes Excel in the same layout as the manual sheets	The layout match lets you compare machine and human row by row
`src/gates/gate0_formulation.py` to `gate4_medical.py`	One module per gate; all pass/fail rules	Gate 2 has no real data behind it
`src/datasets/fsa_client.py`, `companies_house_client.py`,	The three data clients	Two are live calls, one reads a local file

Path	What it is	Notes
`ddinter_client.py`		
`src/data/*.csv`	The tables the rules read: form tiers, dosage bands, conflict pairs, drug interactions, certification requirements	Details in the data-files section
`tests/`	`test_gate0.py` (6 tests), `test_gate4.py` (7), `test_demo_cases.py` (3)	16 in total, all offline
`webapp/`	Flask website: `server.py`, `templates/`, `static/`	Reads the ground truth for its dropdowns
`validate_*.py`	Scripts that compare each gate with the ground truth and print kappa	Read CSVs from the project root
`convert_science_research.py`, `parse_ingredients_key*.py`, `extract_*.py`	Build the data tables from the research team's evidence file	Run once; outputs are already committed
`llm_extract_*.py`, `llm_form_extractor.py`, `run_llm_extraction.py`	The AI extraction layer	Paid API calls; need `ANTHROPIC_API_KEY`
`dsld_*.py`, `usda_filter_supplements.py`, `build_magnesium_dose_comparison.py`	Market-data collection and the magnesium case study	Not needed to run the pipeline
`app.py`, `pages/`, `theme.py`, `home_view.py`	An older Streamlit interface	Retired; safe to delete
`requirements.txt`, `.env.example`, `.vscode/`	Dependencies, key template, editor settings	See the setup section for gaps

Set up and run from a clean machine

You need Python 3.10 or newer (the author used 3.14 on Windows), Git, and a free Companies House API key.

1. Clone and create an environment

```
git clone <repository-url>
cd gate-analysis-pipeline
python -m venv .venv
```

Activate it with `.venv\Scripts\activate` on Windows or `source .venv/bin/activate` on macOS and Linux. Never move or copy a `.venv` folder, because it stores its own path and breaks; delete it and recreate it instead.

2. Install the packages

```
pip install -r requirements.txt
pip install flask
pip install anthropic
```

`requirements.txt` covers the pipeline only. Flask is needed only for the website, and `anthropic` only for the extraction scripts.

3. Set your keys

The code reads environment variables and does **not** load a `.env` file, even though `python-dotenv` is installed. Set the key in the terminal you will run from, every session:

```
$env:COMPANIES_HOUSE_API_KEY="your-key"        # Windows PowerShell
export COMPANIES_HOUSE_API_KEY=your-key         # macOS and Linux
```

Only `COMPANIES_HOUSE_API_KEY` is needed to run the pipeline, demo and website. `ANTHROPIC_API_KEY` is needed only to re-run the extraction scripts. The FSA, DDInter and NIH sources need no key.

4. Run the tests (offline)

```
python -m pytest tests -q
```

Expect `16 passed`; no internet is used.

5. Run the demo (needs internet)

```
python run_demo.py
```

Expect 21 lines starting `ok`, then "Every case behaved as expected", and a workbook at `output/CN_Automated_Output.xlsx`. A line starting `!!` means a live call failed. Close the workbook in Excel before every re-run.

6. Run the website

```
cd webapp
python server.py
```

Open `http://127.0.0.1:5000`. Its brand and product dropdowns read `CN_Final_Corrected.xlsx` from the project root (sheet "Gate 0 — Formulation", headers on row 3); without that file those lists are empty.

If something goes wrong

Symptom	Cause	Fix
`ModuleNotFoundError`	Environment not active, or a package missing	Activate `.venv`; run the install step
Gate 3 fails with "COMPANIES_HOUSE_API_KEY	Key not set in this terminal	Set it, then restart the server

Symptom	Cause	Fix
not set"		
`PermissionError` writing the workbook	File open in Excel	Close it and re-run
A demo case shows `!!`	A live call failed, or a fictional brand matched a real company	Re-run; check the key and connection; rename the brand
`TemplateNotFound` on the website	`templates/` not inside `webapp/`, or run from the wrong folder	Keep the folder layout; run `server.py` from inside `webapp/`

Data sources: which sites feed what

At run time the pipeline needs two keyless FSA services, one keyed Companies House service and a few local tables; every other source was used to build the data or to study the market.

Source	Used for	How it is accessed	Key?	Licence	Role
FSA Regulated Products API, Novel Food register	Gate 1: does an ingredient need novel-food authorisation	One list of about 700 substances at `data.food.gov.uk/regulated-products/id/regime/novel-foods.json`, fetched once per run and matched locally	No	Open Government Licence v3	Live at run time
FSA Food Alerts API	Gate 3: recalls and alerts that mention a brand	`data.food.gov.uk/food-alerts/id` with `search`, `_limit` and `_view` parameters	No	Open Government Licence v3	Live at run time
UK Companies House API	Gate 3: is the company real and active	`api.company-information.service.gov.uk/search/companies?q=` with HTTP Basic auth	Yes, free	Open government data; check the current terms	Live at run time
DDInter drug-interaction database	Gate 4: interactions between supplements and drugs	A local file, `src/data/ddsi_cache.csv`, 334 records filtered from DDInter's bulk downloads (version 1.0; version 2.0 offers	No	Public access	Static copy, refreshed by hand

Source	Used for	How it is accessed	Key?	Licence	Role
		no bulk download)			
NIH Dietary Supplement Label Database (DSLD) API v9	Real product label data for dose analysis and the magnesium case study	`api.ods.od.nih.gov/dsld/v9/search-filter` via `dsld_api_pull.py`	No	CC0	Data preparation only
USDA FoodData Central, Branded Foods	Product ingredient text for context	Bulk CSV download of about 900 MB, filtered locally by `usda_filter_supplements.py`	No	US government public domain	Data preparation only
NIH Office of Dietary Supplements fact sheets	Background on doses and forms	Read and extracted by hand	No	US government public domain	Evidence background
PubMed and Europe PMC	Literature on doses and forms	Open REST APIs	No	Varies by article	Evaluated for Gate 4; not wired into the pipeline

What has no open source at all. Batch lab certificates (Gate 2); advertising rulings from the Advertising Standards Authority (the FSA alerts cover food-safety recalls only); manufacturing location, customer service and testimonials (a person enters these for Gate 3); and the doses used in cited studies (Gate 4 uses a safety threshold instead).

Attribution. FSA data is released under Open Government Licence v3, which requires you to acknowledge the source; keep that wording if you publish results.

How to call each source yourself

These are the same requests the clients in `src/datasets/` make. Try them in a notebook first; they were correct when tested, but public endpoints can change, and every client raises a clear error when one does.

FSA Novel Food register (no key)

```
import requests
URL = "http://data.food.gov.uk/regulated-products/id/regime/novel-foods.json"
data = requests.get(URL, timeout=15).json()
concepts = data["items"][0]["substanceScheme"][0]["hasTopConcept"]
slugs = [c["@id"].rstrip("/").split("/")[-1] for c in concepts]
print(len(slugs), slugs[:5])
```

This is a browsable list, not a search service: an early version assumed a name-search parameter and every call failed. `check_novel_food_status` matches an ingredient's distinctive words against these slugs, and for a few ingredients it also needs a qualifier from the product name (Zinc matches only if "pidolate" appears).

FSA Food Alerts (no key)

```
r = requests.get("https://data.food.gov.uk/food-alerts/id",
                 params={"search": "Healthspan", "_limit": 20, "_view": "full"},
timeout=15)
items = r.json()["items"]
```

The client keeps only alerts whose title or description mentions a supplement word (supplement, vitamin, capsule, tablet, mineral, softgel, gummies); without that filter a retailer's name matches unrelated grocery recalls. These are food-safety recalls, so the check works per brand, not per product.

Companies House (free key)

```
import os, requests
key = os.environ["COMPANIES_HOUSE_API_KEY"]
r = requests.get("https://api.company-information.service.gov.uk/search/companies",
                 params={"q": "Healthspan Ltd"}, auth=(key, ""), timeout=15)
active = [i for i in r.json()["items"] if i.get("company_status", "").lower() ==
"active"]
```

To get a key, register at `developer.company-information.service.gov.uk`, create an application and add a REST key. The key is the username and the password is empty. The client takes the first active result and does not compare names; see the known-issues section. Check the current documentation for the rate limit.

DDInter (a local file, not a live call)

`check_supplement_drug_interactions(name)` reads `src/data/ddsi_cache.csv`, whose columns are `supplement`, `drug`, `severity`, `mechanism` and `management`. A "Major" severity fails Gate 4.

To refresh it, download the bulk CSVs from the DDInter site (version 1.0 offers downloads), keep the rows whose supplement matches an ingredient name in `form_tiers.csv`, and write those five columns. `ddinter_supplement_relevant.csv` is the filtered intermediate. The one-off build script is not in the repository, so write one if you plan regular refreshes.

NIH DSLD (market data, no key)

```
python dsld_api_pull.py
```

Edit the `INGREDIENTS` list at the top; each ingredient is saved to its own CSV in `dsld_exports/`. This was used for the magnesium case study, not by the pipeline.

AI extraction (paid)

```
python run_llm_extraction.py --sample 50
```

The scripts call the Anthropic API with the model named at the top of each file (`claude-sonnet-4-5` when written). Every row is one paid call, so always run a small sample first. Progress is saved every 25 rows and finished rows are skipped on re-run, so an interrupted run resumes without paying twice. The prompts are in Appendix A of the dissertation. The model only extracts; it never decides.

The data files, and how to update them

The rules read five tables in `src/data/`; two Excel workbooks and some derived CSVs sit outside it, and keeping the evidence apart from the answer key is the most important habit in this project.

File	What it holds	Columns	Origin
`form_tiers.csv`	507 rows: each supplement form graded Superior, Acceptable or Reject, with the uses it suits	ingredient, form, tier, bioavailability_note, use_case, source	The research team's evidence file
`dosage_bands.csv`	108 rows: dose limits per ingredient and population	ingredient, population, unit, min_effective_dose, optimal_low, optimal_high, upper_limit, source	The same evidence file
`contraindicated_pairs.csv`	21 rows: ingredient pairs that conflict	ingredient_a, ingredient_b, issue, source	Extracted from the evidence file, plus curated rows
`recommended_testing.csv`	101 rows: the certification level each ingredient needs; 55 need BRCGS-level	ingredient, recommended_testing, requires_brcgs_tier	One column of the evidence file
`ddsi_cache.csv`	334 supplement and drug interaction records	supplement, drug, severity, mechanism, management	DDInter

Outside `src/data/`

- `science_research.xlsx` is the research team's original evidence file, the input to the `convert_*` and `extract_*` scripts.
- `CN_Final_Corrected.xlsx` is the ground truth: one sheet per gate. The Gate 0 sheet has 1,858 rows, of which 104 are category summaries, leaving 1,754 products. It is used only for validation and for the website's dropdowns, never as a pipeline input.
- `parsed_all_categories_ground_truth.csv` (1,754 products with brand, product name, detected form, dose and the manual result), `gate1_clean.csv` (814 rows), `gate3_clean.csv` (281 rows) and `llm_extraction_checkpoint.csv` (the cached AI extraction results) are derived from them.

Provenance. The evidence file has no row-level citations, and rows tagged `needs_citation` have no traceable reference. Treat the tables as a strong draft that needs source checking before anyone relies on a single row.

To add or change an ingredient

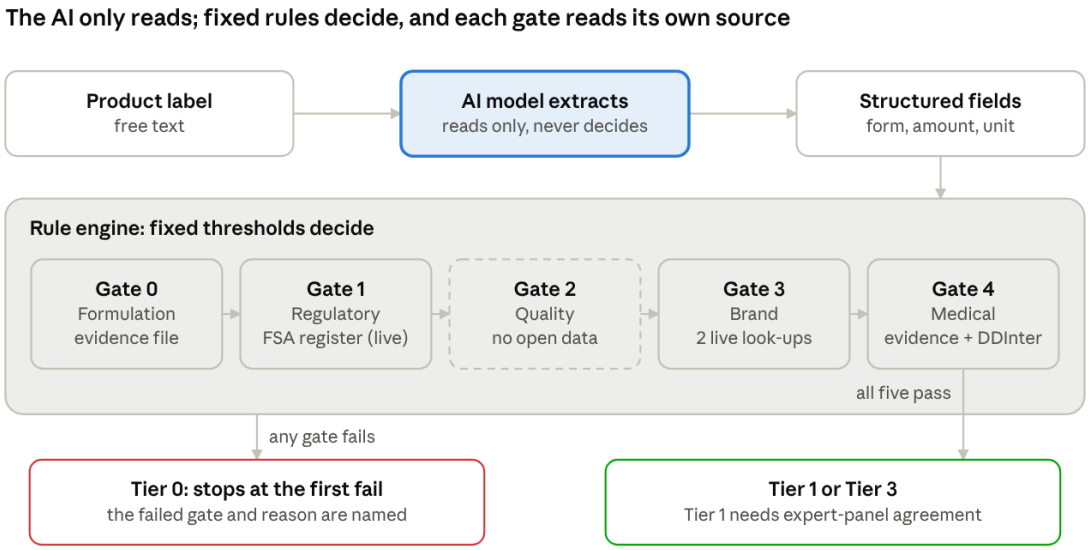
1. Add its forms, tiers and uses to `form_tiers.csv`, and its dose limits to `dosage_bands.csv`.

2. Add a row to `recommended_testing.csv` saying whether BRCGS-level certification is required.
3. Spell the ingredient identically in every file; the lookups match on the name.
4. Re-run `python -m pytest tests -q` and `python run_demo.py`.

Ownership. The two workbooks belong to the research team, and the CSV tables derive from the evidence file. Confirm who may receive and reuse them before this repository changes hands.

How the pipeline works

Every decision is a fixed rule over structured fields; the AI model only produces those fields, and each gate reads its own data source.



the pipeline · 3 steps in, 5 gates with their sources, 2 outcomes

Read it top to bottom. A label goes in as free text, the AI model extracts the form, amount and unit, and from then on only fixed thresholds run. A product moves through the gates in order and stops at the first one it fails; if it clears all five it earns a tier. Gate 2 is dashed because it has no open data behind it.

The boundary to protect. Gates take plain structured fields (ingredient name, form, amount, unit) and return a verdict with a written reason. Keep the AI on the left of that boundary: if a model ever decides pass or fail, the traceability that makes this design defensible is gone.

How to extend it

- **A new check in an existing gate:** add a function that returns a pass flag and a message, call it from that gate's `run_gate` function, and add a test.
- **A new data source:** keep the return shape of the existing client. For example, a real DDInter API can replace the local file as long as `check_supplement_drug_interactions` still returns `found` and a list of interactions.
- **A new gate:** add its inputs and result types in `src/schemas.py`, a module in `src/gates/`, its place in `src/pipeline.py` and a sheet in `report_writer.py`.

- **Tier rules:** `assign_tier` in `gate4_medical.py` decides Tier 1, 3 or 0. Tier 2 can never occur today, because Gate 2 already requires an accredited laboratory.

Validation: what was measured, and how to re-run it

Four gates were compared with hand-made labels using Cohen's kappa; the best, Gate 1, reached moderate agreement, and every tested gate covers only part of its framework.

Gate	Compared (n)	Agreement	Kappa	Scope
0 Formulation	648	70.7%	0.352	3 of 5 parameters
1 Regulatory	766	96.5%	0.412	Novel Food check only
2 Quality	none	none	none	Not tested; no open data
3 History	280	91.4%	-0.033	1 of 5 parameters
3 Disclosure	281	93.2%	-0.013	1 of 5 parameters
4 Medical	145	75.9%	0.153	Dose proxy

Kappa is agreement with the chance matches removed: zero means no better than guessing, and 0.41 to 0.60 is called moderate. It matters here because most labels share one answer; an always-PASS check scores 88.4% agreement on Gate 4.

What is proven. The pipeline runs end to end; each rule can both pass and fail products (the 21-case demo and 16 tests show this); and the live look-ups work.

What is not proven. Accuracy on unseen data (fixes were tuned against the same labels); anything about Gate 2; Gate 4's real parameter, which needs cited-study doses; agreement between the two human reviewers; and the correctness of every row of the evidence file.

The safety-relevant weakness. Where the manual review said FAIL, the engine sometimes passed the product: 117 of 239 on Gate 0, 25 of 35 novel foods on Gate 1, and 11 of 18 on Gate 4.

Re-running the checks

Run these from the project root; they read files from the working directory.

Script	What it checks	Reads	Writes
`validate_gate1_novelfood.py`	Gate 1 against the manual novel-food labels; makes live FSA calls	`gate1_clean.csv`	`gate1_novelfood_comparison.csv`
`validate_gate3_regulatory.py`	Gate 3 FSA alerts check; live calls	`gate3_clean.csv`	`gate3_regulatory_comparison.csv`
`validate_gate3_disclosure.py`	Gate 3 company check; live calls, needs the	`gate3_clean.csv`	`gate3_disclosure_comparison.csv`

Script	What it checks	Reads	Writes
	Companies House key		
`validate_llm_accuracy.py`	Gate 0 form, dose and transparency	`llm_extraction_checkpoint.csv` and the `*_from_science_research.csv` tables	`llm_accuracy_comparison.csv`
`validate_llm_accuracy_gate4.py`	Gate 4 form and dose	The same inputs	`llm_accuracy_comparison.csv`
`validate_dosage_accuracy.py`	An early, regex-based dose check	`parsed_all_categories_ground_truth.csv`, `science_research.xlsx`	`dosage_accuracy_comparison.csv`

Expect results to drift slightly: the registers are live, so a rerun can differ from the figures above. As a sanity check, Gate 1 should land near 96.5% and kappa 0.412 on 766 rows. The Gate 0 and Gate 4 scripts must exclude rows with no matched form, and must not count them as passes; counting them was a real bug that inflated two early results. Also note that `validate_llm_accuracy.py` and `validate_llm_accuracy_gate4.py` both write `llm_accuracy_comparison.csv`, so the second run overwrites the first; copy the file between runs.

Operating the website and the demo

Both exist to show the pipeline working; neither is a production service.

The website

It has four pages: **Home** (a summary and the five gate cards), **Try a Product** (runs the real pipeline on one product), **Accuracy and Validation** (the results and iteration charts) and **Methodology** (how it works and what it does not claim).

- **Try a Product** fills its dropdowns from the evidence tables and the ground truth, so brand and product lists need `CN_Final_Corrected.xlsx`. The Advanced section lets you change the Gate 1 inputs (certifier, batch tracking, disease claim), the Gate 2 heavy-metal readings, and a free-text statement that Gate 3 searches for a company name.
- **What the page fixes for you:** one ingredient at a time; no proprietary blend; customer service always passes; potency always matches the label. So it cannot show the Gate 0 conflict or blend checks, and a clean pass always lands on Tier 3.
- **The Accuracy page draws its charts from a public JavaScript library,** so it needs internet access to show them.
- **It is not deployed.** If you host it, keep the Companies House key in the server's environment variables, never in the code, and remember that free hosting tiers sleep when idle.

The 21-case demo

`run_demo.py` runs 21 hand-built products, each a clean product with exactly one thing changed, so every stop has one clear reason. They narrow from 21 to 15, 12, 9, 6 and 4 cases through the gates. The Demo Guide sheet lists each case with its expected and actual result.

- The failing quality and marketing cases use clearly fictional brands. The demo checks that such a brand does not match a real company on Companies House, and flags the case if it does.
- `tests/test_demo_cases.py` re-runs all 21 cases offline, with only the live look-ups stubbed, so it fails if the demo and the rules drift apart.
- **To add a case:** call `build_case(...)` in `run_demo.py`, add a row to `DEMO_CASES` with its expected stop gate and tier, and update the funnel counts in `test_demo_cases.py`.

Known issues and gotchas

These are the traps a new owner is most likely to hit; most are documented limits of the data, not bugs, and each has a suggested fix.

Issue	What happens	Suggested fix
Company check takes the first active result	A made-up brand can be "verified" against an unrelated company, because the returned name is never compared with the one searched	Add a name-similarity threshold in `check_company_exists`
Tier 2 cannot occur	Gate 2 already requires an accredited lab, and "premium testing" is that same flag	Redefine Tier 2, or remove it
Novel Food check misses species-specific cases	25 of Gate 1's 27 errors sit in five categories (Cordyceps, Omega-3, Astaxanthin, Krill oil, Fulvic acid), where the answer depends on the species	Capture species or source in the product data
FSA alerts are recalls only	Advertising rulings are not covered, and the check is per brand, not per product	Add another source for advertising rulings
Gate 4 uses proxies	The claim check reuses Gate 0's use-case lookup, and the dose check is a safety threshold, not a cited-study dose	Build a cited-study dose table
Evidence-file data quirks	Turmeric (Curcumin) has an upper limit of 210 mg below its 500 mg minimum, so no dose can pass Gate 0; Glucosamine Sulfate's rejected form name sits inside the superior one, so it passes	Review these rows with the research team
Potency tolerance	The stricter 5% limit for a few critical ingredients never matches category names such as "Iron (Bisglycinate)", so 10% applies	Match on a normalised ingredient name
DDInter is a	Only Iron, Magnesium and Magnesium Citrate carry	Request bulk access and

Issue	What happens	Suggested fix
frozen copy	Major entries; new interactions need a manual refresh	automate the refresh
Two validation scripts share an output file	Gate 0 and Gate 4 both write `llm_accuracy_comparison.csv`	Copy the file between runs, or rename the outputs
A Python pitfall	A missing value counts as true in a plain `if`; this silently passed unmatched rows in two scripts	Test for missing values explicitly
A moved virtual environment	It stores its own path and fails with confusing errors	Recreate it; keep one copy of the project
Public endpoints change	The FSA register URL uses `http`; any live call can fail	A failed look-up returns FAIL with "route to human review"; check the client's error text

The README is stale. It still says the pipeline is "empirically-validated, not a prototype", that a lookup beat the AI "across every gate", that there are 6 tests and 5 demo cases, that the interaction data comes from DDInter 2.0, and `.env.example` points to a `src/extraction/` folder that is not in the repository layout. Treat this guide as the current description and update the README before sharing the repository.

Security, licences and permissions

Agree three things before the repository changes hands: who holds the keys, who owns the data, and which licences govern the sources.

- **Keys.** Never commit a key. `.env` is excluded in `.gitignore`, but the code reads environment variables anyway, so keys belong in the terminal or the server settings. Each person should register their own free Companies House key. If a key is ever pasted into chat, a file or a commit, revoke it in the Companies House developer hub and issue a new one.
- **Data ownership.** `science_research.xlsx` and `CN_Final_Corrected.xlsx` belong to the research team, and the CSV tables in `src/data/` derive from the evidence file. Get written confirmation of who may receive, reuse and publish them. Keep the repository private and add people through Settings, then Collaborators.
- **Open-data terms.** The FSA sources are Open Government Licence v3 (acknowledge the source). The NIH DSLD is CC0, and the USDA data is US public domain. Check the current Companies House terms, and check DDInter's redistribution terms before shipping the interaction cache beyond research use.
- **AI use.** The extraction scripts send product label text, which is public, to the Anthropic API, and cost money per row. No personal data is collected anywhere in the project. If you ever run it on private customer data, review your own data-handling rules first.
- **How to describe the output.** Verdicts are screening aids, not certifications, medical advice or legal opinions. Tier 1 is only a candidate that needs expert-panel agreement.

- **Ethics position.** The dissertation states that it uses open public data with no human participants (section 4.8).

A first week, and a roadmap

A new owner can be productive in five days if the first week goes from running things, to understanding them, to checking them.

Day	Goal	Done when
1	Clone, create the environment, run the tests and the demo	`16 passed`, and "Every case behaved as expected"
2	Read this guide and chapters 4 and 5 of the dissertation; run the website	You can explain the pipeline diagram and try a product end to end
3	Call each live source yourself from a notebook	You have seen the FSA register, the alerts and a Companies House result with your own key
4	Re-run the Gate 1 and Gate 3 validation scripts	Gate 1 lands near 96.5% and kappa 0.412
5	Review the known issues and the data-ownership questions	You have agreed priorities and who owns which file

Roadmap, in order of how much each step strengthens the claims

1. **Run the frozen pipeline once on unseen, expert-reviewed cases** and report the result whatever it is.
2. **Add a name-similarity check to the Gate 3 company look-up.**
3. **Build a cited-study dose table** so Gate 4 compares with real study doses.
4. **Add species and source information** so Gate 1 stops missing species-specific novel foods.
5. **Add row-level citations to the evidence file** and review its quirks with the research team.
6. **Find real batch-certificate data for Gate 2,** perhaps through partner laboratories.
7. **Automate the DDInter refresh,** ideally through bulk access from its team.
8. **Tidy the project:** update the README, add Flask and `anthropic` to `requirements.txt`, and load `.env` in code so keys need not be set by hand.

Glossary and credits

Term	Plain meaning
Gate	One yes-or-no check; a product must pass all five in order
Tier	The label for a product that clears every gate: Tier 1 (a candidate needing expert-panel agreement), Tier 3, or Tier 0 for a fail
Extraction versus decision	Extraction turns label text into fields; a decision is the pass or

Term	Plain meaning
	fail, made only by fixed rules
Evidence file	The research team's table of 101 ingredients: good and bad forms, and dose ranges
Ground truth	The hand-made answer key the results are graded against; never a pipeline input
Cohen's kappa	Agreement with the chance matches removed; zero means no better than chance
Skewed labels	When almost all answers are the same, so even a lazy check scores high
Novel Food	An ingredient not widely eaten in the UK or EU before 1997 that needs authorisation to be sold
GMP, BRCGS	Good manufacturing practice; BRCGS is one recognised certification body
ISO 17025	The standard a testing laboratory must meet to be accredited
COA	Certificate of analysis: a lab's report on one batch
DDInter	A public database of drug interactions
Proxy	A stand-in measure used when the real one is unavailable

Keys -
1.COMPANIES_HOUSE_API_KEY="4f8a30d8-a7fc-4ab3-8869-1e2c14dc1856"
2. ANTHROPIC_API_KEY = "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"

“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”